



evanexpert

FASE 1 · ANALYSE & SCOPE

Wat we bouwen en wat niet.

Inventarisatie van de bestaande Smart Buy Store front-end, scope-afbakening, must/should/later, en architectuurkeuzes die het systeem simpel en foutbestendig houden.

PROJECT

Smart Buy Store

DOCUMENT

01 van 10

DATUM

14 mei 2026

AUTEUR

Evan Poul

EvanExpert Studio · KvK 78740320 · info@evanexpert.nl

evanexpert.nl

Wat er nu staat

Live: <https://smart-buy-store.vercel.app> (Next.js op Vercel, productie deploy)

Element	Status
Homepage (hero, USP's, productshowcase, testimonials, nieuwsbrief)	Werkend, visueel sterk
Navigatie (categorieën in menu)	Menu-items zichtbaar — sommige categorie-pagina's 404 op live deploy (mismatch met source)
Productdetailpagina's	Bestaan in source code
Winkelmand <code>/winkelwagen</code>	Bestaat niet als route — alleen client-side <code>useCart</code> hook
Account <code>/account</code>	Niet aanwezig in source — Header linkt er wel naar
Checkout / betalingen	Niet geïmplementeerd
Footer-bedrijfsgegevens	Letterlijk <code>TODO:</code> voor e-mail, telefoon, adres, BTW
Bedrijfsinfo gevonden	KvK 42000760, betaalmethoden in footer: iDEAL, Klarna, Visa, Mastercard

Source code-inventaris

Stack (Next.js 14 App Router, geen backend):

Onderdeel	Versie	Doel
Next.js	14.2.18	App Router, SSR/SSG mogelijk
React	18	UI
TypeScript	5	Type-safety
Tailwind	3.4	Styling
Radix UI	dialog, hover-card, tabs, toast	Toegankelijke primitives
Framer Motion	11	Animaties
Lucide	icons	Iconenset
Backend / DB / Auth / Mollie	geen	Nog te bouwen

Pagina's die in source bestaan:

Route	Status
/ — homepage	Compleet, visueel sterk
/categorie/[slug]	Bestaat in code
/product/[slug]	Bestaat in code
/aanbiedingen	Bestaat
/winkel	Bestaat
/over-ons	Bestaat
/account	Niet bestaand (Header linkt ernaar)
/winkelwagen	Niet bestaand (Header linkt ernaar)
/checkout	Niet bestaand

Data laag (volledig statisch): - 8 producten hardcoded in `data/products.ts` - 5 categorieën hardcoded (`koelkasten` , `wasmachines` , `vaatwassers` , `koken` , `drogers`) - Merken hardcoded - Cart leeft alleen in `localStorage` (`hooks/useCart.ts`) — geen server-side persistence

Wat goed gepositioneerd is voor de back-end: - Strakke component-architectuur (`home/`, `product/`, `layout/`, `ui/`) - `Product` -type goed gedefinieerd in `types/product.ts` — we breiden uit, gooien niet weg - Tailwind config + design tokens al opgezet - `useCart` -hook heeft de juiste shape — alleen storage-backend wisselen - App Router betekent Server Components + Server Actions — geen aparte API-layer nodig

Wat het systeem minimaal moet kunnen

Voor de klant: - Producten doorbladeren per categorie + product details bekijken - Winkelmand + checkout - Bestelling plaatsen → afrekenen via Mollie → bevestiging - Account aanmaken (eigen e-mail) en inloggen - Eigen bestellingen + facturen inzien/downloaden

Voor de eigenaren (adminkant): - Producten beheren (toevoegen, aanpassen, verbergen) - Categorieën beheren - Bestellingen zien + status aanpassen - Klanten zien (zonder kunnen verwijderen) - Facturen downloaden - E-mailtemplates aanpassen - Kortingscodes beheren

Onderhuids (verplicht aanwezig, onzichtbaar): - Mollie webhook = enige bron-van-waarheid voor "betaald" - Automatische factuur-PDF na betaling - Automatische e-mails (bevestiging, status, factuur) - Idempotentie: nooit dubbele facturen, dubbele e-mails

Scope — must / should / later

Must-have (FASE 1–9, voor go-live)

- Productcatalogus met categorieën
- Winkelmand + checkout-flow
- Mollie betalingen (iDEAL primair, andere methoden uit Mollie-dashboard)
- Klantaccount: registreren, inloggen, wachtwoord vergeten, gegevens, adressen, bestellingen, facturen
- Adminportaal: dashboard, bestellingen, klanten, producten, categorieën, kortingscodes, facturen, e-mailtemplates, instellingen
- Rollen: admin / medewerker / bezorger-planning / klant
- Automatische facturen (PDF, snapshot, opvolgend nummer)
- Automatische e-mails per status-overgang
- Soft-delete patronen voor product/categorie/klant
- Audit trail op bestellingen (statuslogboek)

Should-have (FASE 9–10 of vlak na go-live)

- Voorraadbeheer (simpel: op voorraad / niet op voorraad — geen multi-warehouse, geen reserveringen)
- Eenvoudige zoekfunctie op productpagina's
- Kortingscode-types: percentage óf vast bedrag (geen complexe regels)
- Bezorgafpraak/datum-keuze in checkout
- Tracking-tabblad voor bezorgrol — overzicht van wat vandaag/morgen bezorgd moet worden

Later (na go-live)

- Productvarianten (kleur/maat) — voor witgoed nu niet nodig
- Productreviews (echte reviews ipv statische testimonials)
- Wishlists / favorieten
- Loyaliteitsprogramma
- Boekhoudkoppeling (CSV-export als start)
- Verzendkoppelingen (PostNL/DHL API) — Smart Buy bezorgt zelf
- Multi-language
- A/B testing, marketing automation, segmenten

Architectuurkeuzes om het simpel en foutbestendig te houden

Keuze	Waarom
Eén Next.js app voor klant én admin	Eén deploy, één domein, één auth-systeem. Admin-routes gated op <code>/admin/*</code> met role-check.
Supabase als backend (Postgres + Auth + RLS + Storage)	Auth voor klanten én admins uit dezelfde tabel met rol-veld. Storage voor factuur-PDFs en productfoto's. RLS voorkomt dat klanten elkaars data zien — beveiliging op DB-niveau.
Mollie webhook = enige waarheid voor "betaald"	UI ververset betaalstatus alleen via DB-query, nooit via redirect. Voorkomt fake-paid via URL-manipulatie.
Idempotentie via Mollie payment ID	Webhook-events opgeslagen met Mollie ID. Tweede call = no-op. Voorkomt dubbele facturen + e-mails.
Facturen = onveranderlijke snapshots	Bij betaling worden klantgegevens, adres, regels, prijzen, BTW en bedrijfsgegevens als JSON opgeslagen. PDF gerenderd daaruit. Eigenaar kan factuurregels niet bewerken — wettelijk juist.
Soft delete / archive ipv hard delete	Klanten met bestellingen worden nooit verwijderd. Producten op <code>verborgen=true</code> . Categorieën op <code>actief=false</code> . Behoud van geschiedenis.
Bevestiging op risicovolle acties	Statuswijzigingen, annuleringen, kortingscode > 25% = modal met expliciete bevestiging.
Geen handmatige status "Betaald"	UI biedt deze knop niet. Betaalstatus is read-only voor admins. Bezorgstatus is wel handmatig.
Centraal e-mail via Resend (of SMTP via env)	One provider, audit-log per verzonden mail. Idempotentie op <code>(order_id, event_type)</code> .
Audit trail op orders	Elke statuswijziging logt: wie, wanneer, oude → nieuwe status.
Productfoto's via Supabase Storage	Eén opslag voor uploads.

Wat ik bewust NIET ga bouwen

- Een echt CRM (leads, pipelines, segmenten). Smart Buy heeft klanten + bestellingen — dat is geen CRM, dat is een order-overzicht.

- Aparte admin-frontend / aparte hosting. Te complex om te onderhouden.
- Microservices. Eén Next.js app + Supabase = klaar.
- Eigen betaalstroom. Mollie doet alles wat nodig is.
- Eigen authenticatie. Supabase Auth voorkomt classes of bugs.

Beslissingen vastgelegd

Beslissing	Gevolg voor architectuur
Mollie bestaat nog niet	In FASE 7 staat als eerste stap "Mollie-account opzetten + API-key in <code>.env</code> ". Code wordt environment-driven (test/live key via <code>MOLLIE_API_KEY</code>).
BTW 21% regulier	BTW-nummer is wettelijk verplicht op iedere factuur. Smart Buy heeft het BTW-nummer (of vraagt aan) — moet vóór FASE 8 bekend zijn.
E-mail provider TBD	Mail-laag geabstraheerd in één <code>lib/mail.ts</code> . Onder de motorkap: keuze tussen Resend (default) of generieke SMTP via env-var <code>MAIL_PROVIDER=resent smtp</code> .
Bezorging met postcode-check	Klant kiest bezorgdatum. Same-day delivery alleen bij postcode in 6 provincies , anders reguliere bezorging. Postcode 4-cijferige prefix → provincie lookup (statische JSON, geen externe API), allowlist beheerbaar via Instellingen → Bezorging.

Same-day allowlist

Friesland · Groningen · Drenthe · Overijssel · Flevoland · Gelderland

Acceptatiecriteria (uit briefing)

- Een niet-technische eigenaar kan binnen 5 minuten een product toevoegen
- Een eigenaar ziet direct welke bestellingen betaald zijn en bezorgd moeten worden
- Een bestelling wordt pas betaald na Mollie webhook
- Automatische factuur wordt gemaakt na betaling
- Automatische bevestigingsmail met factuur wordt verstuurd na betaling
- Klant kan account maken en eigen facturen downloaden
- Eigenaren kunnen geen grote fouten maken door verkeerd te klikken
- Het systeem is volledig Nederlandstalig

Akkoord op deze fase ontvangen. Doorgegaan naar FASE 2 — Technisch plan.